

# Exercício (Novo): API de Finanças — "Meu Controle de Gastos"

---

**Turma:** LionsDev

**Tópicos:** Boilerplate, rotas protegidas com JWT, dono do recurso via token, validação de valor no service, **cálculo de resumo em JavaScript** (entradas, saídas e saldo), Query Params e status codes.

**Nível:** intermediário. A novidade aqui é uma rota de **resumo** que calcula totais a partir das suas transações.

---

## 1. Contexto

Você vai criar uma API de **controle financeiro pessoal**. Cada usuário registra suas **entradas** (salário, vendas) e **saídas** (contas, compras), e pode pedir um **resumo** com total de entradas, total de saídas e o **saldo**. Cada pessoa só enxerga o próprio dinheiro.

Para este exercício, trabalhe com valores em **reais** usando **Number** (ex.: **150.5**). Não precisa usar centavos.

---

## 2. Ponto de Partida: o Boilerplate

Partimos do **boilerplate LionsDev**: <https://github.com/nicolassmotta/lionsdev-boilerplate>

1. Clone, `npm install`, crie o `.env` (`MONGO_URI`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `BCRYPT_SALT_ROUNDS`) e suba o servidor.
2. Já estão prontos: camada de `Usuario`, cadastro/login com `bcrypt/JWT`, middleware `autenticar` (preenche `req.usuario = { id, email }`), `criarErro` e o middleware de erro.

**Antes de começar, pegue seu token** e envie `Authorization: Bearer SEU_TOKEN` em todas as rotas novas.

---

## 3. Regras de Ouro

1. **Toda rota é protegida.** `router.use(authenticar)` no topo do arquivo de rotas.
2. **O dono vem do token** ( `req.usuario.id` ), **nunca do body**.
3. **Toda consulta filtra pelo dono** ( `{ _id: idTransacao, usuario: idDoUsuario }` ).  
Se não achar → **404** .
4. **Listagem nunca dá 404:** sem transações, devolva array vazio (e o resumo vem zerado).

---

## 4. Arquivos que você vai criar

| Camada     | Arquivo                                               |
|------------|-------------------------------------------------------|
| Model      | <code>src/models/transacao.model.js</code>            |
| Repository | <code>src/repositories/transacao.repository.js</code> |
| Service    | <code>src/services/transacao.service.js</code>        |
| Controller | <code>src/controllers/transacao.controller.js</code>  |
| Routes     | <code>src/routes/transacao.routes.js</code>           |

---

## 5. Etapa 1 — Model **Transacao**

- `descricao`: `String`, obrigatório, `trim` .
- `tipo`: `String`, obrigatório, `enum`: `["entrada", "saida"]` .
- `categoria`: `String`, opcional, `trim` (ex.: `"salário"`, `"mercado"`, `"lazer"` ).
- `valor`: `Number`, obrigatório, `min`: `[0.01, "0 valor deve ser maior que zero."]` .
- `data`: `String`, opcional (ex.: `"2026-06-17"` ).
- `usuario`: `ObjectId`, `ref`: `"Usuario"`, obrigatório.
- `timestamps`: `true` .

**Critérios de aceite:** `descricao`, `tipo`, `valor` e `usuario` são obrigatórios; `tipo` só aceita `entrada` ou `saida`; `valor` não aceita zero nem negativo.

### Conceito novo — `enum`, `min` numérico e o campo de dono ( `ObjectId` + `ref` )

- `enum` trava `tipo` a uma lista fechada ( `"entrada"` / `"saida"` ); qualquer outro valor vira **400** .
- `min` valida um número mínimo já no Schema (sem `if`): `valor`: `{ type: Number, required: true, min: [0.01, "0 valor deve ser maior que zero."] }` .
- `usuario` guarda o **dono** da transação — uma referência ao `_id` de um `Usuario` :

```
import mongoose from "mongoose";
// ...dentro do Schema
tipo: { type: String, required: true, enum: ["entrada", "saida"] },
valor: { type: Number, required: true, min: [0.01, "O valor deve ser maior que zero."] },
usuario: { type: mongoose.Schema.Types.ObjectId, ref: "Usuario", required: true },
```

## 6. Etapa 2 — Repository

- `criar(dados) → Transacao.create(dados)`.
- `listarPorUsuario(idUsuario, filtro = {}) → Transacao.find({ usuario: idUsuario, ...filtro }).sort({ createdAt: -1 })`.
- `buscarPorIdDoDono(idTransacao, idUsuario) → Transacao.findOne({ _id: idTransacao, usuario: idUsuario })`.
- `atualizarPorIdDoDono(idTransacao, idUsuario, dados) → findOneAndUpdate(filtro, dados, { new: true, runValidators: true })`.
- `deletarPorIdDoDono(idTransacao, idUsuario) → findOneAndDelete(filtro)`.
- Exporte tudo no objeto `TransacaoRepository`.

## 7. Etapa 3 — Service

- `registrar(idDoUsuario, dados)` : além da validação do Schema, **reforce** que **valor** é um número maior que zero antes de salvar ( `if (!dados.valor || dados.valor <= 0) throw criarErro("O valor deve ser maior que zero.", 400)` ). Monta o objeto com `usuario: idDoUsuario` e cria.
- `listarMinhas(idDoUsuario)` : retorna a lista do usuário.
- `buscarMinha(idDoUsuario, idTransacao)` : `buscarPorIdDoDono(...)` ; se `null` → `404` .
- `atualizarMinha(idDoUsuario, idTransacao, dados)` : `atualizarPorIdDoDono(...)` ; se `null` → `404` .
- `removerMinha(idDoUsuario, idTransacao)` : `deletarPorIdDoDono(...)` ; se `null` → `404` ; se ok → `{ message: "Transação removida com sucesso." }` .
- `resumoDoUsuario(idDoUsuario)` : a parte nova. Busque **todas** as transações do usuário e calcule, em JavaScript:
  - `totalEntradas` = soma dos `valor` onde `tipo === "entrada"` .

- `totalSaidas` = soma dos `valor` onde `tipo === "saida"`.
- `saldo` = `totalEntradas - totalSaidas`.
- Retorne `{ totalEntradas, totalSaidas, saldo, quantidade }`.

Use `filter` + `reduce` ou um simples `for...of` para somar. Sem transações, tudo retorna `0`.

### Conceito novo — calcular o resumo em JavaScript ( `filter` + `reduce` )

O banco te devolve um **array** de transações. Os totais são calculados na mão, em JS, dentro do service:

```
const transacoes = await
TransacaoRepository.listarPorUsuario(idDoUsuario);

const totalEntradas = transacoes
  .filter((t) => t.tipo === "entrada") // só as entradas
  .reduce((soma, t) => soma + t.valor, 0); // soma os valores (começa em 0)

const totalSaidas = transacoes
  .filter((t) => t.tipo === "saida")
  .reduce((soma, t) => soma + t.valor, 0);

const saldo = totalEntradas - totalSaidas;
return { totalEntradas, totalSaidas, saldo, quantidade:
transacoes.length };
```

- `.filter(fn)` devolve um novo array só com os itens que passam no teste.
- `.reduce((acumulador, item) => ..., inicial)` "espreme" o array num único valor — aqui, a soma. O `0` no final é o ponto de partida (sem transações → soma `0`, nunca dá erro).

## 8. Etapa 4 — Controller e Etapa 5 — Routes

Crie o controller ( `try/catch` + `next(error)` ) e as rotas. No

`src/routes/transacao.routes.js`, com `router.use(authenticar)` no topo:

- **POST /** → registrar ( **201** )
- **GET /** → listarMinhas ( **200** com { **transacoes** } )
- **GET /resumo** → resumo ( **200** ) — **declare antes de GET /:id**
- **GET /:id** → buscarMinha ( **200** )
- **PATCH /:id** → atualizarMinha ( **200** )
- **DELETE /:id** → removerMinha ( **200** )

No **src/app.js** , **antes** do middleware 404:

```
import transacaoRoutes from "../routes/transacao.routes.js";
app.use("/api/transacoes", transacaoRoutes);
```

**Atenção à ordem das rotas:** se **GET /:id** vier antes de **GET /resumo** , o Express vai entender "resumo" como um **:id** e quebrar a rota de resumo.

## 9. Rotas finais

| Método | Rota                          | Proteção | Descrição                         |
|--------|-------------------------------|----------|-----------------------------------|
| POST   | <b>/api/transacoes</b>        | JWT      | Registrar entrada ou saída        |
| GET    | <b>/api/transacoes</b>        | JWT      | Listar minhas transações          |
| GET    | <b>/api/transacoes/resumo</b> | JWT      | Total de entradas, saídas e saldo |
| GET    | <b>/api/transacoes/:id</b>    | JWT      | Ver uma transação minha           |
| PATCH  | <b>/api/transacoes/:id</b>    | JWT      | Editar uma transação minha        |
| DELETE | <b>/api/transacoes/:id</b>    | JWT      | Remover uma transação minha       |

## 10. Fluxo de Testes (use o **requests.http**)

1. Cadastro/login → copie o **token** .
2. **POST /api/transacoes** com { **"descricao": "Salário", "tipo": "entrada", "valor": 3000** } → **201** .
3. **POST** com { **"descricao": "Mercado", "tipo": "saida", "valor": 450.5** } → **201** .
4. **POST** com { **"descricao": "Erro", "tipo": "saida", "valor": 0** } → **400** .

5. `GET /api/transacoes/resumo` → `totalEntradas` 3000, `totalSaidas` 450.5, `saldo` 2549.5.
  6. `GET /api/transacoes` → só as suas.
  7. `DELETE /api/transacoes/:id` → `200`; repita → `404`.
  8. Com um **segundo usuário**, peça `/api/transacoes/resumo` → vem zerado (não enxerga o dinheiro do primeiro).
  9. Qualquer rota **sem token** → `401`.
- 

## 11. Desafios Bônus

1. `GET /api/transacoes?tipo=saida` — filtra **entre as suas** por tipo (use `req.query.tipo`).
2. No resumo, adicione `totalPorCategoria` (um objeto com a soma das saídas por `categoria`).
3. `GET /api/transacoes?de=2026-06-01&ate=2026-06-30` — filtra suas transações por intervalo de `data`.

**Como ler um Query Param ( `?tipo=saida` ):** no controller, leia de `req.query` e repasse ao service, **sempre** somando ao filtro do dono para não vaziar dados de outras pessoas:

```
// controller
const filtro = {};
if (req.query.tipo) filtro.tipo = req.query.tipo;
const transacoes = await TransacaoService.listarMinhas(req.usuario.id,
filtro);
// no repository: Transacao.find({ usuario: idUsuario, ...filtro })
```